# Software Architecture and Design Specification

Project: Blood Bank Management System (BBMS)

Version: 1.0

Authors: Team 9 (Avaneesh Ganesh Aroor, Avrit Sharma, Ayush Kumar, Ayushman Dey)

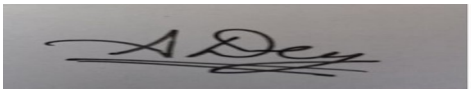
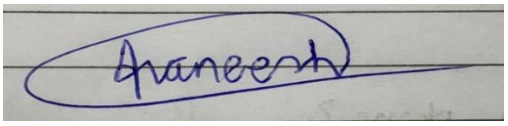
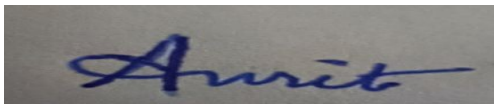
Date: 04-10-2026

Status: Draft

## Revision History

| Version | Date | Author | Change Summary |
|---|---|---|---|
| 1.0 | 04-10-2026 | Team 9 | Initial architecture and design specification, derived from SRS v1.0 (03-09-2026) and STP v1.0 (17-09-2026). Defines the design specification IDs (DS-xxx) referenced by the SRS RTM. |

## Approvals

| Role | Name | Signature/Date |
|---|---|---|
| Product Owner | Ayushman Dey (PES2UG24CS102) |  |
| QA Lead | Avaneesh Ganesh Aroor (PES2UG24CS098) |  |
| Dev Lead | Avrit Sharma (PES2UG24CS100) |  |
| Test Engineer | Ayush Kumar (PES2UG24CS101) |  |
| Course Coordinator | Prof. Asha Kumari | |

## 1. Introduction

### 1.1 Purpose

This document specifies the architecture and design of the Blood Bank Management System (BBMS). It explains how the system is structured, how its parts interact, and why the main design choices were made, so that the

functional, non-functional and security requirements in the SRS can be built and verified. It also gives the testers the design information they need to execute the Software Test Plan (STP).

### 1.2 Scope

The design covers the full application scoped in the SRS: user registration and login, donor and donation management, blood inventory tracking, blood requests and fulfillment, notifications and reports, administration and audit, and the security and access-control layer that spans all modules.

Consistent with the SRS and STP, the design does not cover laboratory equipment internals, external hospital systems beyond the interfaces defined here, the internal delivery pipeline of any third-party email or SMS provider, or the reliability of the database engine itself.

### 1.3 Audience

Developers, QA engineers, blood bank staff representatives, administrators, security reviewers, project evaluators, and future maintainers of the system.

### 1.4 Definitions

BBMS (Blood Bank Management System), ADR (Architecture Decision Record), API (Application Programming Interface), RBAC (Role-Based Access Control), ORM (Object-Relational Mapper), DRF (Django REST Framework), TLS (Transport Layer Security), JWT (JSON Web Token), RPO / RTO (Recovery Point / Time Objective), FEFO (First-Expired, First-Out), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), DS (Design Specification), RTM (Requirements Traceability Matrix), SRS (Software Requirements Specification), STP (Software Test Plan). A fuller glossary is in Appendix 5.1.

## 2. Document Overview

### 2.1 How to use this document

Section 3 describes the architecture: goals, stakeholders, the component and deployment views, the chosen pattern with its decision records, the technology stack, risks, requirement traceability, and the security architecture. Section 4 describes the detailed design: data model and domain rules, sequence diagrams for the four most important flows, the REST API, error handling, logging and monitoring, UX design, and open issues. Section 5 holds the glossary, references and tools.

Every requirement in the SRS is mapped to a design specification ID (DS-001 to DS-SR-005) in Section 3.8. These are the same IDs that appear in the "Section ref / Design Spec" column of the SRS RTM, so the RTM can now be completed end to end: requirement, design, test case.

### 2.2 Related Documents

| Document | Version / Date | Relationship to this document |
|---|---|---|
| Software Requirements Specification (SRS), Team 9 | v1.0, 03-09-2026 | Source of all functional (BBMS-F), non-functional (BBMS-NF) and security (BBMS-SR) requirements realised here. |
| Software Test Plan (STP), Team 9 | v1.0, 17-09-2026 | Defines how this design is verified. Section 3.8.2 lists the testability hooks the design provides for it. |
| Requirements Traceability Matrix (SRS Section 8) | N / not run | Design Spec column is populated by Section 3.8 of this document. |

# 3. Architecture

## 3.1 Goals & Constraints

### *Architectural goals*

| ID | Goal (quality attribute) | Target | SRS source |
|---|---|---|---|
| G1 | Safety and correctness of blood issuance | No incompatible, expired or double-issued unit can ever be issued; stock always equals the sum of recorded transactions. | F-022, F-032, F-034 |
| G2 | Security and privacy of personal data | TLS 1.2+, deny-by-default RBAC, hashed passwords, input sanitisation, session expiry. | NF-003, SR-001 to SR-005 |
| G3 | Traceability and auditability | Every login and every critical change leaves a protected, searchable audit record kept at least 1 year. | F-051, NF-004, SR-004 |
| G4 | Performance | 95% of normal search and dashboard requests complete within 3 seconds. | NF-001 |
| G5 | Availability and recoverability | 99.5% monthly availability (about 3.6 hours of unplanned downtime per 30 days); RPO 24 h; RTO 4 h. | NF-002, NF-006 |
| G6 | Usability and accessibility | Responsive UI, labelled forms, keyboard navigation, readable error messages. | NF-005 |
| G7 | Maintainability and testability | Clear module boundaries a four-person team can own; domain rules testable without a database; tooling matches the STP. | STP Sections 5 and 6 |

### *Constraints*

- **Technology:** the SRS allows Python (Django) or JavaScript (Node.js) with a relational database (PostgreSQL or MySQL). This design selects Django and PostgreSQL (ADR-002, ADR-003).
- **Platform:** web application reached over HTTPS from desktop and mobile browsers; deployable on a standard Linux server or cloud VM.
- **Data protection:** personal and transaction data must be protected, roles enforced on the server, and inventory kept accurate and free of duplicate or inconsistent records.
- **Users:** staff have varying levels of technical experience, so workflows must be simple and forgiving.
- **Project:** four team members with overlapping coursework, and a fixed calendar (STP Section 7: test execution 03-Oct to 13-Oct-2026, UAT 14-Oct to 16-Oct-2026).

## 3.2 Stakeholders & Concerns

| Stakeholder | Main concerns | Addressed in |
|---|---|---|
| Donor | Easy booking, privacy of health and contact data, accurate donation history. | 3.9, 4.3, 4.5 |
| Recipient / Requester | Quick request submission, clear status, reliable availability information. | 4.1.2, 4.3, 4.5 |
| Blood Bank Staff | Accurate stock, simple verification and issue workflow, early low-stock warning, no expired units shown. | 3.5, 4.1, 4.2 |
| Administrator | Control of accounts and roles, configuration, audit oversight, reports. | 3.9, 4.3, 4.4 |
| Medical / safety reviewers | Compatibility rules enforced without exception; donor eligibility respected. | 4.1.3, 3.7 |
| Developers and maintainers | Modularity, clear boundaries, easy local setup, documented APIs. | 3.5, 3.6, 4.1 |

| Stakeholder | Main concerns | Addressed in |
|---|---|---|
| QA team | Testable design, controllable time, seed data, sandbox notifications. | 3.8.2 |
| Evaluators / Course Coordinator | Traceability from requirement to design to test; clear decision rationale. | 3.5, 3.8 |

## 3.3 Component (UML) Diagram

Figure 1 shows the logical components and their dependencies. Requests flow from the browser through the Nginx reverse proxy into the API layer, where every request passes the security layer (authentication, RBAC, throttling). The API layer calls one of six domain services. All persistence goes through the data access layer into PostgreSQL. Background processing (the daily expiry job and the notification dispatcher) runs in a separate worker process that uses the same services and data access layer.

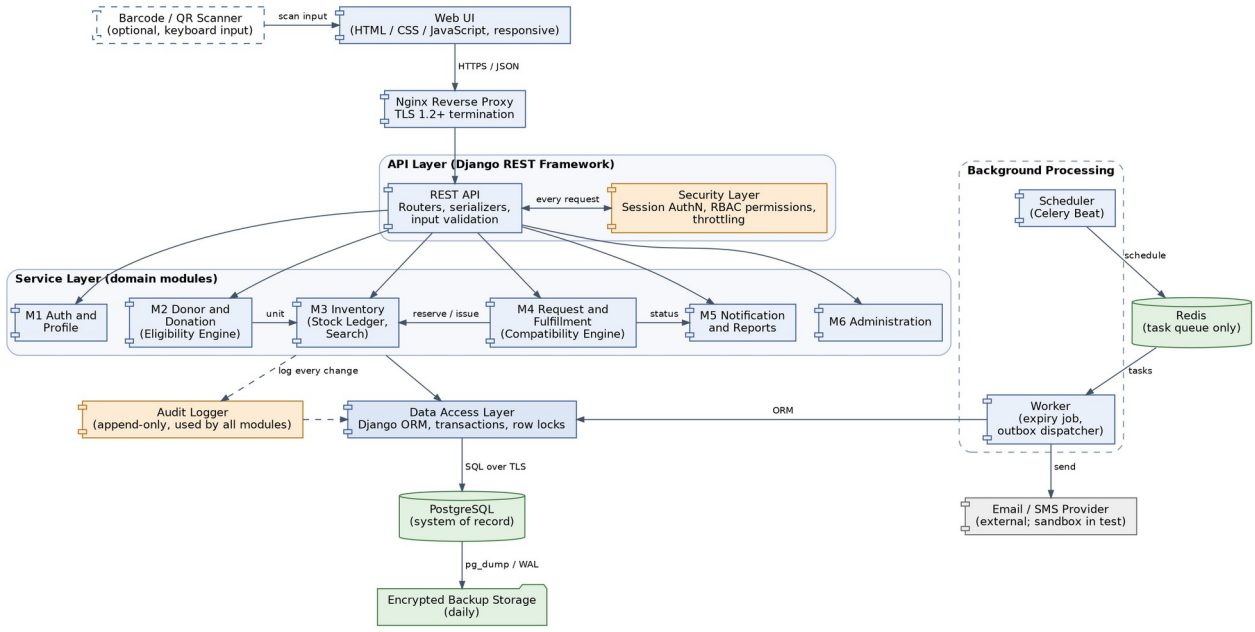



*Figure 1. BBMS component diagram (logical view)*

## 3.4 Component Descriptions

| Component | Responsibility | Realises |
|---|---|---|
| Web UI | Responsive HTML/CSS/JavaScript front end with role-specific dashboards. Validates input for convenience only; the server re-validates everything. Accepts barcode/QR scanner input as ordinary keyboard text in the unit-code field. | NF-005, SRS 3.1, 3.2 |
| Nginx reverse proxy | Terminates TLS 1.2+, redirects HTTP to HTTPS, sets security headers, serves static files, forwards to the application server. | SR-001 |
| API Layer (DRF) | Routes /api/v1 calls, deserialises and validates requests, maps domain errors to HTTP status codes and the standard error format, and applies permission classes. | SR-003, SRS 3.4 |
| Security Layer | Session authentication, RBAC permission classes (deny by default), ownership checks, login throttling, session idle and absolute timeout, CSRF protection. | F-002, F-003, SR-002, SR-005 |
| M1 Auth and Profile Service | Registration, login and logout, password hashing, profile view and update, role lookup. | F-001 to F-004, NF-003 |
| M2 Donor and Donation Service | Donation slots, appointment booking, duplicate prevention, donation recording, donation history. Contains the Eligibility Engine (age, weight, interval). | F-010 to F-013 |

| Component | Responsibility | Realises |
|---|---|---|
| M3 Inventory Service | Blood unit creation and validation, stock ledger, status transitions, availability search, expiry handling, stock counts and threshold checks. | F-020 to F-024, F-041 |
| M4 Request and Fulfillment Service | Request submission, review (approve, reject, hold), reservation, issue, request history. Contains the Compatibility Engine (ABO and Rh rules). | F-030 to F-034 |
| M5 Notification and Report Service | Writes notification outbox rows, dispatches email and SMS through a provider adapter, logs failures, generates reports. | F-040, F-041, F-042 |
| M6 Administration Service | Staff account creation, disabling and role assignment, stock threshold configuration, audit log search, delivery failure review. | F-050, F-051 |
| Audit Logger | Writes append-only audit records inside the same database transaction as the change being audited. Used by every module. | F-051, NF-004, SR-004 |
| Data Access Layer | Django ORM models, transactions, row-level locking, parameterised queries. The only code that talks to the database. | F-022, SR-003 |
| Scheduler and Worker | Celery Beat triggers the daily expiry job; the Celery worker runs it and also dispatches pending notifications. | F-024, F-040 |
| PostgreSQL | System of record for all business, audit and job data. TLS-protected, reachable only from the application and worker hosts. | All |
| Redis | Task queue for Celery only. It holds no business data; losing it loses no records because the outbox lives in PostgreSQL. | F-024, F-040 |
| Email / SMS provider adapter | Small interface with a real provider implementation and a console/mock implementation for test. | F-040, STP Section 11 |
| Backup storage | Receives the daily encrypted database dump, kept offsite from the database host. | NF-006 |

## 3.5 Chosen Architecture Pattern and Rationale

BBMS uses a **layered architecture built as a modular monolith**: one Django project deployed as a single unit, divided into presentation, API, service (domain) and data access layers, with each functional area (M1 to M6) as its own module with a defined interface. A separate worker process runs scheduled and asynchronous work from the same code base.

**Why this fits.** Blood issuance needs strong consistency: reserving a unit, writing the ledger entry, updating the request and writing the audit record must succeed or fail together. A single database transaction gives that directly. A modular monolith also suits a four-person team (clear module ownership, one deployment, one test environment) and the stated scale of a single blood bank.

**Rejected alternatives.** Microservices were rejected because they add network failure modes, distributed transactions and operational overhead that would endanger inventory accuracy and exceed the team's capacity. A fully event-sourced design was rejected as unnecessary; instead an append-only stock ledger and audit log capture the history that matters.

### *Architecture Decision Records*

| ID | Decision | Alternatives considered | Rationale |
|---|---|---|---|
| ADR-001 | Layered modular monolith | Microservices; serverless functions | Strong transactional consistency for stock; one deployable; matches team size and scale (G1, G7). |
| ADR-002 | Python 3.12 with Django and Django REST Framework | Node.js (Express/NestJS) | Built-in ORM, migrations, admin tooling, auth, CSRF and Argon2 support reduce custom security code. Pytest fits the STP. SRS permits either stack. |
| ADR-003 | PostgreSQL as system of record | MySQL | Partial unique indexes (duplicate appointments, one active allocation per unit), SELECT ... FOR UPDATE SKIP LOCKED, and strong constraint support. |

| ID | Decision | Alternatives considered | Rationale |
|---|---|---|---|
| ADR-004 | Server-side sessions in HttpOnly, Secure, SameSite cookies | JWT access and refresh tokens | Idle timeout (SR-005) and immediate revocation when an account is disabled (F-050) are trivial with server-side sessions and awkward with stateless JWTs. JWT remains an option if an external hospital API client is added later. |
| ADR-005 | Stock is derived from unit status, backed by an append-only inventory ledger; changes use pessimistic row locks | A mutable stock counter per group | A counter can drift. Deriving stock from units makes drift impossible, and the ledger gives the audit trail for F-022. Row locks prevent two staff reserving the same unit. |
| ADR-006 | Transactional outbox for notifications | Send email or SMS inside the request | The notification row commits with the status change, so none is lost. Provider outages cannot block or fail clinical workflows, and failures are logged and retried (SRS 3.4). |
| ADR-007 | Celery Beat plus worker for the daily expiry job | OS cron calling a script | Shares the same code, logging and monitoring as the app. Supports retries and a manual trigger in test (STP Section 5.2). Cron remains an acceptable fallback. |
| ADR-008 | Audit log as an append-only database table | Log files only; third-party log service | Searchable by actor, action, time and record (NF-004). The application database role has INSERT and SELECT only, so ordinary code cannot alter history. |

## 3.6 Technology Stack & Data Stores

| Layer / concern | Choice | Notes |
|---|---|---|
| Language and framework | Python 3.12+, Django 5.2 LTS, Django REST Framework | ADR-002. |
| Front end | HTML5, CSS3, JavaScript, Bootstrap 5; Django templates with fetch() calls to the JSON API | Responsive grid; no heavy SPA framework needed for the scope. |
| Database | PostgreSQL 16+ | ADR-003. TLS enabled, private network only. |
| Background work | Celery with Celery Beat; Redis as broker | ADR-007. Redis holds queue messages only. |
| Web and app server | Nginx in front of Gunicorn | TLS 1.2+, HSTS, security headers. |
| Passwords | Argon2 via argon2-cffi (Django Argon2PasswordHasher) | One-way hashing with per-password salt (NF-003). |
| Sessions | Django database-backed sessions with CSRF protection | ADR-004. Idle and absolute timeouts configurable. |
| API documentation | OpenAPI schema via drf-spectacular, shown in Swagger UI | Doubles as the Postman collection source for the STP. |
| Logging and monitoring | Structured JSON application logs; Prometheus-style metrics and an external uptime probe | See Section 4.4. |
| Testing tools | Pytest with pytest-django, Selenium, Postman, JMeter | As listed in STP Section 6. |
| Source control and defects | Git and GitHub, GitHub Issues | Defect tracking per STP Section 6. |
| Backups | pg_dump plus optional WAL archiving to encrypted offsite storage | NF-006. |

### *Data stores*

| Store | Contents | Notes |
|---|---|---|
| PostgreSQL | Users and profiles, donation slots, appointments, donations, blood units, inventory ledger, requests, allocations, notifications (outbox), stock thresholds, audit log, job runs. | System of record. Single source of truth for all business data. |
| Redis | Celery task messages. | Disposable. Rebuilt from the outbox if lost. |
| Offsite backup storage | Daily encrypted database dumps. | Retention period to be set by the team (suggested 30 days). |

| Store | Contents | Notes |
|---|---|---|
| Application log files | Structured operational logs with no passwords, tokens or clinical detail. | Rotated; separate from the audit log. |

### *Deployment view*

Figure 2 shows the deployment. The application server hosts Nginx, the Django application under Gunicorn, the Celery worker and Beat, and Redis. PostgreSQL runs on a separate host or managed service reachable only over a private network, and daily dumps go to separate encrypted storage. An external probe checks /healthz every minute to measure the 99.5% availability target.

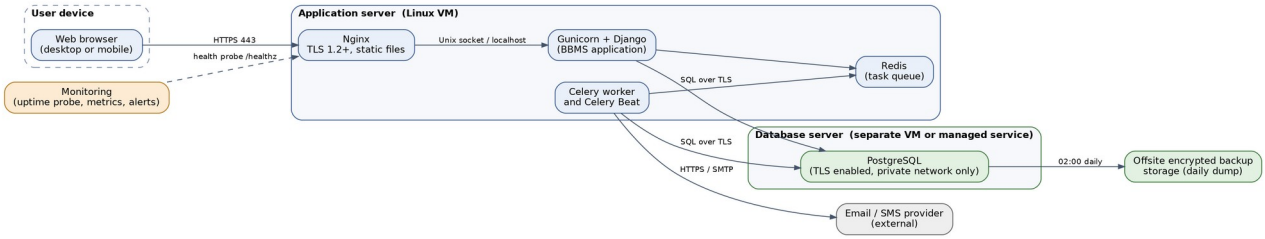



*Figure 2. BBMS deployment view*

## 3.7 Risks & Mitigations

| ID | Risk | Mitigation |
|---|---|---|
| R1 | Two staff reserve or issue the same unit at the same moment, or a retry decrements stock twice. | Row-level locks (SELECT ... FOR UPDATE SKIP LOCKED), a unique constraint allowing only one active allocation per unit, and an Idempotency-Key on the issue call (ADR-005, DS-032). |
| R2 | A blood-type compatibility defect reaches issuance (safety critical). | One Compatibility Engine implemented as a pure function over a lookup table, tested against the full 8x8 matrix early (STP 5.1). It is applied twice: when selecting candidate units and again just before reserving (DS-034). |
| R3 | The daily expiry job fails or is skipped, leaving expired units marked available. | Availability queries also filter on expiry_date, so a missed run cannot expose an expired unit. The job is idempotent and batched, records every run in job_run, and monitoring alerts on a failed or missing run (DS-024). |
| R4 | The email/SMS provider is down or slow. | Transactional outbox with retries and logged failures; clinical workflows never wait on the provider (ADR-006). |
| R5 | Single application host or database host failure threatens the 99.5% target. | Process supervision with automatic restart, separate database host, health probe and alerts, documented restore runbook, and an option to add a second app server behind Nginx. |
| R6 | Unauthorised access to donor or recipient data, or privilege escalation. | Deny-by-default permission classes, object-level ownership checks, TLS everywhere, secure cookies, audit of access-denied events on sensitive operations (Section 3.9). |
| R7 | Search and dashboards exceed 3 seconds as data grows. | Composite index on (blood_group, component, status, expiry_date), pagination, select_related, and JMeter checks against the 95th percentile target (DS-NF-001). |
| R8 | Audit log tampering or uncontrolled growth. | INSERT/SELECT-only database role, monthly table partitions, indexed search fields, retention of at least 1 year (DS-NF-004). |
| R9 | Time-zone and date-boundary mistakes around expiry, eligibility and the daily job. | Timestamps stored in UTC; calendar dates (expiry, DOB, donation date) interpreted in the blood bank's configured time zone; job runs at 00:05 local time; boundary rule defined in Section 4.1.3. |
| R10 | Limited team capacity and overlapping coursework. | Module-per-owner assignment, thin vertical slices, shared domain rules in small pure functions that are quick to test. |

## 3.8 Traceability to Requirements

### *3.8.1 Design specification register*

Each SRS requirement maps to one design specification (DS) ID, the component that realises it, and the main design mechanism. DS IDs follow the numbering already used in the SRS RTM.

| Req ID | Design Spec | Component | Design mechanism |
|---|---|---|---|
| BBMS-F-001 | DS-001 | M1 | Registration endpoint creates Donor or Recipient accounts only. Unique constraints on email and phone. Serializer validation of mandatory fields. Password stored as an Argon2 hash. |
| BBMS-F-002 | DS-002 | M1, Security | Credential check, then a server-side session in an HttpOnly, Secure cookie; user is redirected to the dashboard for their role (Figure 9). |
| BBMS-F-003 | DS-003 | Security | Role on the account; DRF permission class on every view, deny by default; ownership checks for object access; matrix in 3.9.2. |
| BBMS-F-004 | DS-004 | M1 | Profile serializer with a per-role whitelist of writable fields. Role, active flag and verified medical fields are read-only to the user. |
| BBMS-F-010 | DS-010 | M2 | Donation slots with capacity. Booking runs under a slot row lock. Booked appointment shows in donor history (Figure 6). |
| BBMS-F-011 | DS-011 | M2 | Staff-only call creates one donation row that is never updated or deleted through the API (corrections are compensating records). Updates donor last-donation date in the same transaction. |
| BBMS-F-012 | DS-012 | M2, DB | Partial unique index on (donor_id, slot_id) where status is BOOKED, plus a service pre-check. Duplicate returns 409. |
| BBMS-F-013 | DS-013 | M2 Eligibility Engine | Pure function: age at least 18, weight at least 50 kg, at least 56 days since last whole-blood donation. Evaluated at booking and again when staff record the donation. |
| BBMS-F-020 | DS-020 | M3 | Only Staff can create a unit, after verification. Unique unit_code suitable for barcode/QR scanning. Linked to its donation. |
| BBMS-F-021 | DS-021 | M3 | Mandatory columns are NOT NULL. Validators: expiry after collection date, quantity above zero, blood group and component from fixed enumerations. |
| BBMS-F-022 | DS-022 | M3, DAL | Every status change writes an inventory_txn ledger row in the same transaction. Available stock is a count of AVAILABLE, unexpired units. No manual stock editing. |
| BBMS-F-023 | DS-023 | M3 | Query filters status AVAILABLE and expiry date after today. Staff see unit-level results; recipients see aggregate counts only. |
| BBMS-F-024 | DS-024 | Scheduler, Worker, M3 | Daily Celery Beat job, idempotent and batched, recorded in job_run, with an as_of_date parameter for testing (Figure 8). |
| BBMS-F-030 | DS-030 | M4 | Request submission with mandatory patient and requirement fields. Receives a unique request ID and starts in PENDING. |
| BBMS-F-031 | DS-031 | M4, Audit | Request state machine (Figure 4). Reject and hold require a reason. Reviewer and timestamp stored and audited. |
| BBMS-F-032 | DS-032 | M4, M3 | Reserve, then issue, inside locked transactions. FEFO selection. One active allocation per unit. Idempotency-Key makes issue safe to retry (Figure 7). |
| BBMS-F-033 | DS-033 | M4 | List and detail endpoints filtered to the requester. Another user's request returns 404. |
| BBMS-F-034 | DS-034 | M4 Compatibility Engine | Lookup-table function used for candidate selection and re-checked before reserving. Rules in Section 4.1.3. |
| BBMS-F-040 | DS-040 | M5 | Each status transition writes notification outbox rows in the same transaction. The dispatcher sends and logs failures. |
| BBMS-F-041 | DS-041 | M5, M3 | Per group and component threshold, configurable by Admin. Checked after each stock-changing transaction and by the daily job. Alert raised on the downward crossing, once until stock recovers. |
| BBMS-F-042 | DS-042 | M5 | Report queries over database records with date range and filters. Server-side pagination; CSV export optional. |
| BBMS-F-050 | DS-050 | M6 | Admin-only endpoints to create, disable and change roles of staff. Disabling ends the user's sessions immediately. Changes audited. |
| BBMS-F-051 | DS-051 | Audit Logger | Append-only audit_log written in the same transaction as the change. Records actor, action, time, affected record. Readable by Admin only. |
| BBMS-NF-001 | DS-NF-001 | All tiers | Indexes on search columns, pagination, query tuning, gzip at Nginx. Verified with JMeter against 95th percentile at most 3 s. |
| BBMS-NF-002 | DS-NF-002 | Deployment | Process supervision with auto-restart, separate database host, one-minute health probe, planned maintenance windows excluded from the measurement. |

| Req ID | Design Spec | Component | Design mechanism |
|---|---|---|---|
| BBMS-NF-003 | DS-NF-003 | M1, Security | Argon2 hasher configured in Django. Log filters strip passwords and tokens. No plaintext credential anywhere in storage or logs. |
| BBMS-NF-004 | DS-NF-004 | Audit Logger | UTC timestamps, indexes on time, actor and record, retention of at least 1 year, purge only by a controlled job after the retention period. |
| BBMS-NF-005 | DS-NF-005 | Web UI | Responsive layout, labelled form controls, keyboard navigation, readable error messages (Section 4.5). |
| BBMS-NF-006 | DS-NF-006 | Backup | Daily dump at 02:00 to encrypted offsite storage (RPO 24 h). Documented restore runbook and periodic restore drill (RTO 4 h). |
| BBMS-SR-001 | DS-SR-001 | Nginx | TLS 1.2+ only, HTTP redirected to HTTPS, HSTS. Database connections also use TLS. |
| BBMS-SR-002 | DS-SR-002 | Security | Permission class on every view; services re-check the actor's role for sensitive operations; nothing relies on client-side checks. |
| BBMS-SR-003 | DS-SR-003 | API Layer, DAL | Serializer validation, parameterised ORM queries only, template auto-escaping, Content-Security-Policy header. |
| BBMS-SR-004 | DS-SR-004 | Audit Logger | Explicit event catalogue (Section 4.4) including login failures, role changes, inventory issues and approvals. |
| BBMS-SR-005 | DS-SR-005 | Security | Sliding idle timeout (default 20 minutes, configurable) and absolute limit (default 8 hours). Expired session returns 401 `session_expired`. |

### *3.8.2 Testability hooks for the STP*

| STP activity | What the design provides |
|---|---|
| 5.1 Blood type compatibility matrix | `acceptable_donor_groups(recipient_group, component)` and `is_compatible(donor_group, recipient_group, component)` are pure functions, so all 64 ABO/Rh pairings can be unit-tested without a database or the UI. |
| 5.2 Scheduled job testing | The expiry task and its management command accept an `as_of_date`, so expiry boundaries are tested by changing the date instead of waiting. A manual trigger exists in test environments only. Failure mid-run is testable because each batch commits separately. |
| 5.3 Role permission matrix | Table 3.9.2 is the oracle. Permissions are declared per view in one place, so each cell of the matrix maps to one positive and one negative API test. |
| 5.4 Security validation | Session idle timeout is an environment setting (can be set to seconds in test). Audit event catalogue in Section 4.4 lists exactly which events must appear. TLS configuration is isolated in Nginx. |
| 6 Test data | A `seed_test_data` command loads accounts for all four roles, units across all eight blood groups including expired and near-expiry units, and requests in pending, approved, rejected and fulfilled states. |
| 11 Notification sandbox | The provider adapter has a console/mock implementation, so tests assert on the outbox and delivery log without contacting a real provider. |
| NF-001 performance | `/healthz` and request-latency metrics support JMeter runs against the search and dashboard endpoints. |

## 3.9 Security Architecture

### *3.9.1 Threat modelling (STRIDE) and controls*

Trust boundaries: (1) browser to Nginx, (2) Nginx to application, (3) application to database and Redis, (4) worker to the external notification provider.

| STRIDE category | Threat in BBMS | Control | Req |
|---|---|---|---|
| Spoofing | Attacker logs in as a staff member or donor by guessing or reusing credentials. | Argon2 hashed passwords, login throttling, generic login error, HttpOnly Secure SameSite session cookie, session idle timeout. | SR-005, NF-003 |
| Tampering | Altering a request in transit, editing stock directly, changing blood group or a donation record. | TLS 1.2+, server-side validation, stock changes only through services that write ledger rows, immutable donation records, CSRF tokens. | SR-001, SR-003, F-011, F-022 |

| STRIDE category | Threat in BBMS | Control | Req |
|---|---|---|---|
| Repudiation | A user denies approving a request or issuing a unit. | Append-only audit log with actor, action, time and record, written in the same transaction as the change. | F-051, SR-004 |
| Information disclosure | Donor or recipient data exposed through another account, logs or error messages; ID enumeration. | Ownership checks, 404 for foreign objects, aggregate-only availability for recipients, no sensitive data in logs or errors, TLS to the database. | SR-002, F-033 |
| Denial of service | Request flooding, brute-force login, or an overlong notification backlog. | Rate limiting at Nginx and per account, pagination limits, asynchronous notifications, health probe and alerts. | NF-001, NF-002 |
| Elevation of privilege | A donor or recipient calls a staff endpoint, or staff manage accounts. | Deny-by-default RBAC permission class on every view, role changes limited to Admin and audited, public registration can only create Donor or Recipient. | F-003, SR-002, F-050 |

#### *Other security measures*

- **Input handling (SR-003):** schema validation on every endpoint, parameterised ORM queries only (no string-built SQL), output auto-escaping, Content-Security-Policy and X-Content-Type-Options headers.
- **Secrets:** database credentials, secret keys and provider keys come from environment variables or a secrets store, never from source control.
- **Data minimisation:** recipients never see donor identities; availability for recipients is aggregate only; logs exclude clinical detail.
- **Audit protection:** the application database role cannot UPDATE or DELETE audit rows; only Admin can read them through the API.

### *3.9.2 Role permission matrix*

This matrix is the design baseline for BBMS-F-003 and BBMS-SR-002 and the oracle for STP Section 5.3. "Own" means only the user's own records; "Agg" means aggregate counts only; a dash means denied.

| Operation | Donor | Recipient | Staff | Admin |
|---|---|---|---|---|
| Self-register an account | Yes | Yes | - | - |
| Log in, log out, view and edit own profile | Own | Own | Own | Own |
| Book or cancel an appointment | Own | - | - | - |
| View own donation history | Own | - | - | - |
| View any donor and their appointments | - | - | Yes | - |
| Record a completed donation | - | - | Yes | - |
| Create, discard or accept return of a blood unit | - | - | Yes | - |
| Search availability by group and component | - | Agg | Unit level | - |
| Submit a blood request | - | Yes | - | - |
| View own requests and history | - | Own | - | - |
| View all requests | - | - | Yes | - |
| Approve, reject or hold a request | - | - | Yes | - |
| Reserve and issue units | - | - | Yes | - |
| View own notifications | Own | Own | Own | Own |
| Generate reports | - | - | Yes | Yes |
| Configure low-stock thresholds | - | - | - | Yes |
| Create, disable and change roles of staff accounts | - | - | - | Yes |
| View audit log and notification delivery failures | - | - | - | Yes |

**Design assumption A-3:** administrators manage accounts, configuration, reports and audit but do not perform operational actions such as approving requests or issuing units. This separates duties and simplifies the negative tests. It needs confirmation from the Product Owner (see Open Issues, Section 4.6).

# 4. Design

## 4.1 Design Overview

BBMS is designed as layered modules that separate concerns and keep the safety-critical rules small and testable. Four rules apply across the whole code base:

1. **Thin views, fat services.** API views only authenticate, validate and translate. All business rules live in the service layer.
2. **One transaction per business action.** A state change, its ledger row, its audit row and its notification outbox rows commit together or not at all.
3. **Deny by default.** A view without an explicit permission class is rejected. Services re-check the actor's role for sensitive actions.
4. **Pure domain functions.** Eligibility and compatibility are plain functions with no database access, so they are tested exhaustively and quickly.

| Django app (package) | Module | Contents |
|---|---|---|
| accounts | M1 Auth and Profile | User and profile models, registration, login, session handling, profile API. |
| donors | M2 Donor and Donation | Slots, appointments, donations, eligibility engine, booking service. |
| inventory | M3 Inventory | Blood units, inventory ledger, search, expiry task, threshold check. |
| requests | M4 Request and Fulfillment | Requests, allocations, compatibility engine, decision, reserve and issue services. |
| notifications | M5 Notifications and Reports | Outbox, provider adapter, dispatcher task, report queries. |
| adminops | M6 Administration | Staff management, threshold configuration, audit and failure views. |
| audit | Audit Logger | Audit model and record() helper, called inside services. |
| core | Shared | Permission classes, error handler, enumerations, settings, health check. |

### *4.1.1 Data model*

Figure 3 shows the main entities and their relationships. All primary keys are UUIDs; user-facing identifiers (unit_code, request number such as REQ-2026-000123) are separate, unique columns.

| Entity | Purpose | Key constraints and notes |
|---|---|---|
| USER | Account for every person using the system. | Unique email and phone. One role (DONOR, RECIPIENT, STAFF, ADMIN). is_active flag. Password stored only as hash. |
| DONOR_PROFILE | Donor health and eligibility data. | Date of birth, weight, blood group, last_donation_date (updated only when a donation is recorded). |
| DONATION_SLOT | A bookable time slot. | capacity greater than zero; bookings counted under a row lock. |
| APPOINTMENT | A donor's booking for a slot. | Status BOOKED, CANCELLED, COMPLETED, NO_SHOW. Partial unique index on (donor, slot) where BOOKED. |
| DONATION | Completed donation record. | Immutable: no update or delete path in the API. Corrections are new compensating records. |
| BLOOD_UNIT | One verified unit in inventory. | Unique unit_code. Group and component from enumerations. expiry_date after collection date. Status per Figure 5. Index on (group, component, status, expiry_date). |
| INVENTORY_TXN | Append-only stock ledger. | One row per RESERVE, RELEASE, ISSUE, EXPIRE, DISCARD, RETURN, with actor and time. Never updated. |
| BLOOD_REQUEST | A request for blood. | Status per Figure 4. Decision reason mandatory for REJECT and HOLD. Reviewer and review time stored. |
| UNIT_ALLOCATION | Link between a request and a reserved unit. | Partial unique index: at most one active allocation per unit. |
| NOTIFICATION | Outbox of messages to send. | Status PENDING, SENT, FAILED. retry_count and last_error recorded for failed deliveries. |

| Entity | Purpose | Key constraints and notes |
|---|---|---|
| AUDIT_LOG | Protected record of critical events. | Append-only. Actor, role, action, entity type and id, outcome, IP, correlation id, UTC time. |
| STOCK_THRESHOLD | Low-stock level per group and component. | Configurable by Admin. alert_active flag prevents repeated alerts. |
| JOB_RUN | History of scheduled job executions. | Unique per (job name, run date) so runs cannot overlap. Status, counts and error text. |

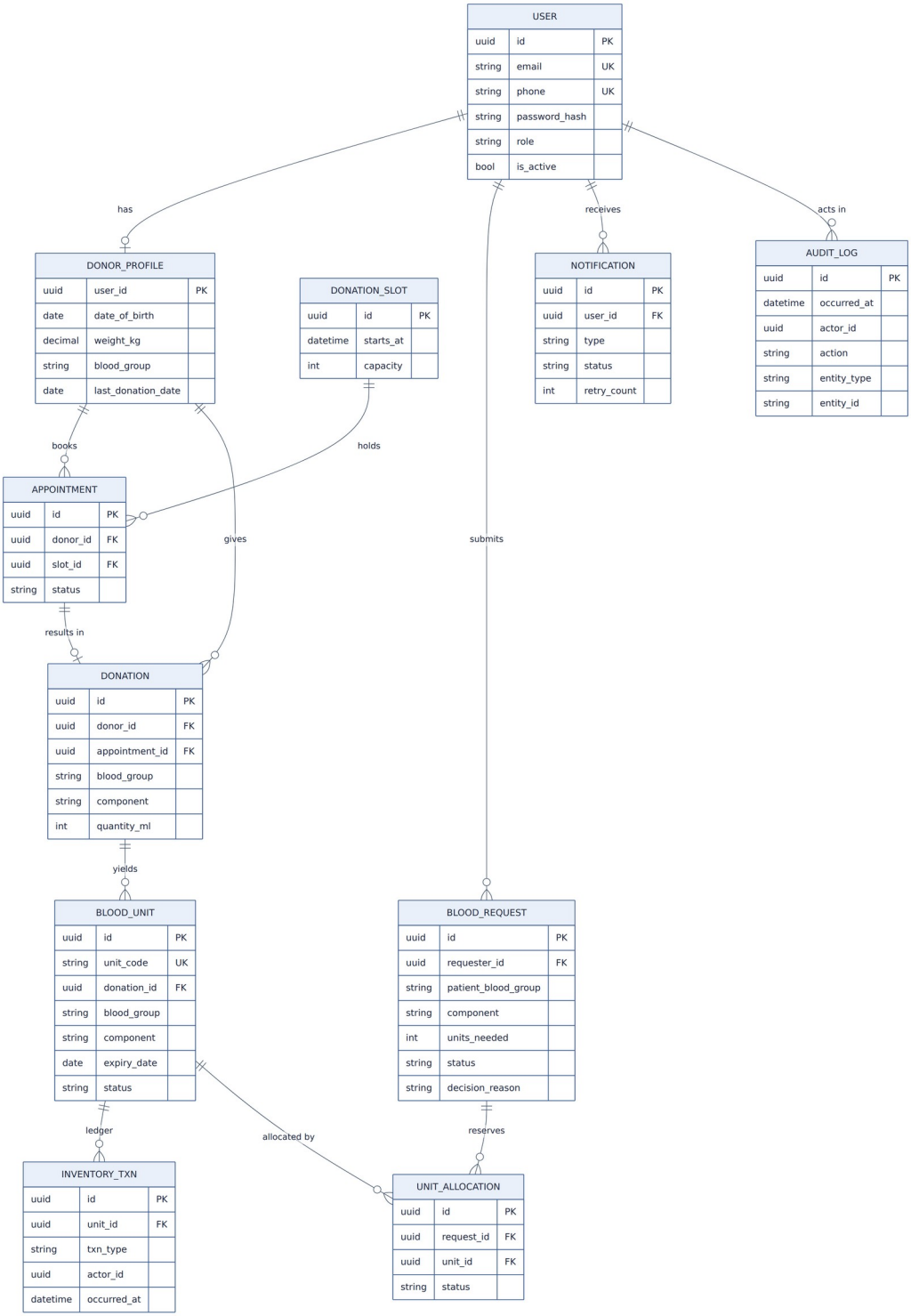



*Figure 3. BBMS entity-relationship diagram (key attributes only)*

### *4.1.2 Lifecycle state models*

Blood requests and blood units each follow a fixed state machine. Any transition not shown is rejected with 409 invalid_state_transition.

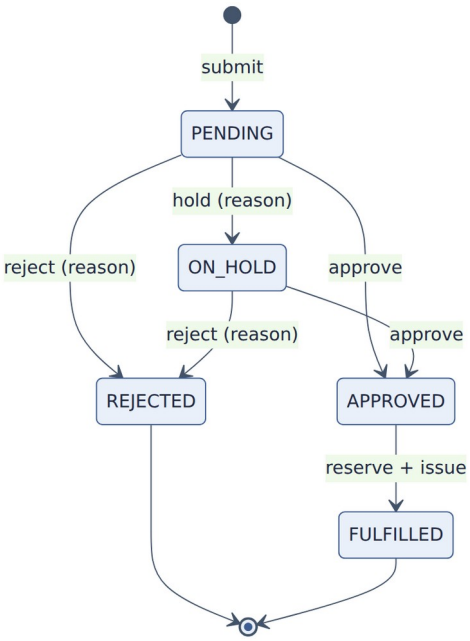



*Figure 4. Blood request lifecycle*

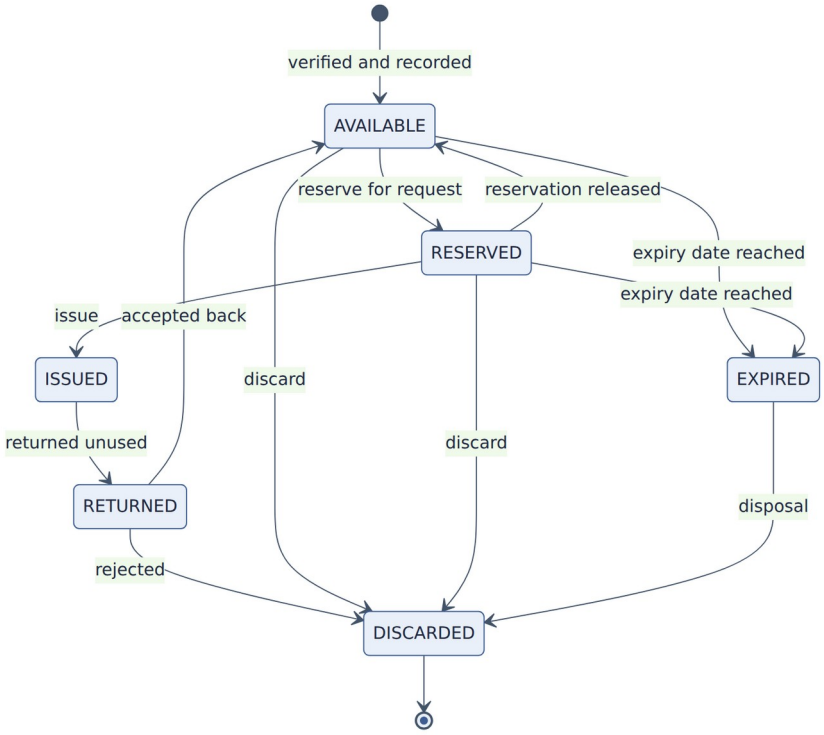



*Figure 5. Blood unit lifecycle*

### *4.1.3 Domain rules*

**Donor eligibility (DS-013).** A donor may book only if all three rules hold on the appointment date:

- age is at least 18 years;
- weight is at least 50 kg;
- at least 56 days have passed since the last completed whole-blood donation.

The check returns every failed rule with a readable reason and, for the interval rule, the next eligible date. It runs again when staff record the donation, using the weight measured at screening.

**Blood type compatibility (DS-034).** Rules from SRS BBMS-F-034: O- is the universal donor, AB+ is the universal recipient, an Rh- recipient can receive Rh- only, and an Rh+ recipient can receive both. The resulting table of acceptable donor groups per recipient group:

| Recipient group | Acceptable donor groups |
|---|---|
| **O-** | O- |
| **O+** | O-, O+ |
| **A-** | A-, O- |
| **A+** | A-, A+, O-, O+ |
| **B-** | B-, O- |
| **B+** | B-, B+, O-, O+ |
| **AB-** | AB-, A-, B-, O- |
| **AB+** | All eight groups |

The function takes the component as a parameter. Version 1.0 applies this rule to every component, as the SRS states it. Plasma and platelets follow different ABO rules in practice, so this is raised as Open Issue OI-1 for medical review.

#### *Other rules*

- **Expiry rule (DS-024):** a unit whose expiry_date is today or earlier is expired. The daily job marks it EXPIRED, and availability queries independently exclude it, so correctness does not depend on the job having run.
- **Allocation policy (DS-032):** among compatible, available units, the earliest expiry date is chosen first (FEFO), preferring an exact group match over O- to conserve the universal donor.
- **Low-stock rule (DS-041):** after every transaction that lowers stock, if the count for a group and component has fallen below its threshold and no alert is active, one alert is raised. The flag clears when stock returns to or above the threshold.
- **Immutability (DS-011):** donation and ledger rows are insert-only.
- **Enumerations:** blood groups A+, A-, B+, B-, AB+, AB-, O+, O-; components WHOLE_BLOOD, RBC, PLASMA, PLATELETS; urgency ROUTINE, URGENT, EMERGENCY.

### 4.2 UML Sequence Diagrams

Four flows are shown, chosen because they exercise the highest-risk requirements: donation booking with eligibility and duplicate prevention (F-010, F-012, F-013), request approval with compatibility-checked reservation and issue (F-031, F-032, F-034), the scheduled expiry job with low-stock alerting and notification delivery (F-024, F-040, F-041), and login with session expiry (F-002, SR-004, SR-005).

### *Flow 1: Schedule a donation appointment*

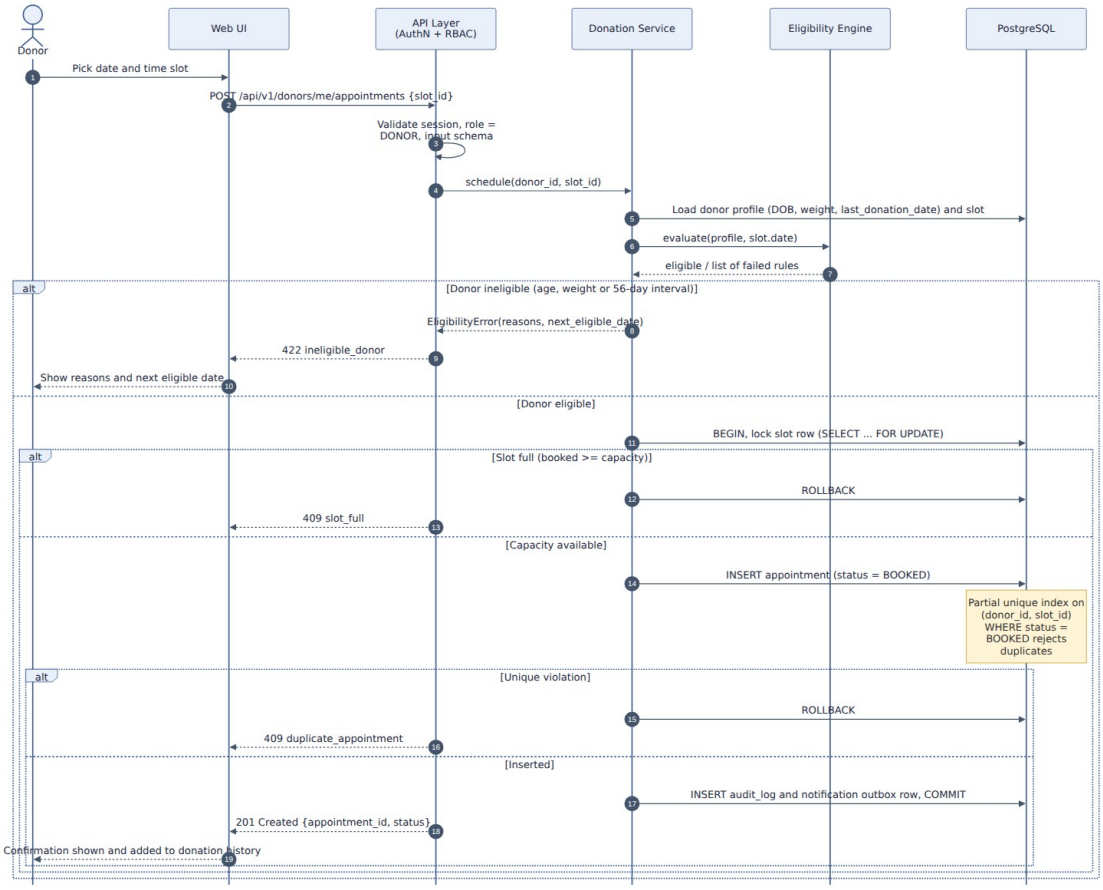



*Figure 6. Sequence diagram: schedule a donation appointment*

### *Flow 2: Approve, reserve and issue blood for a request*

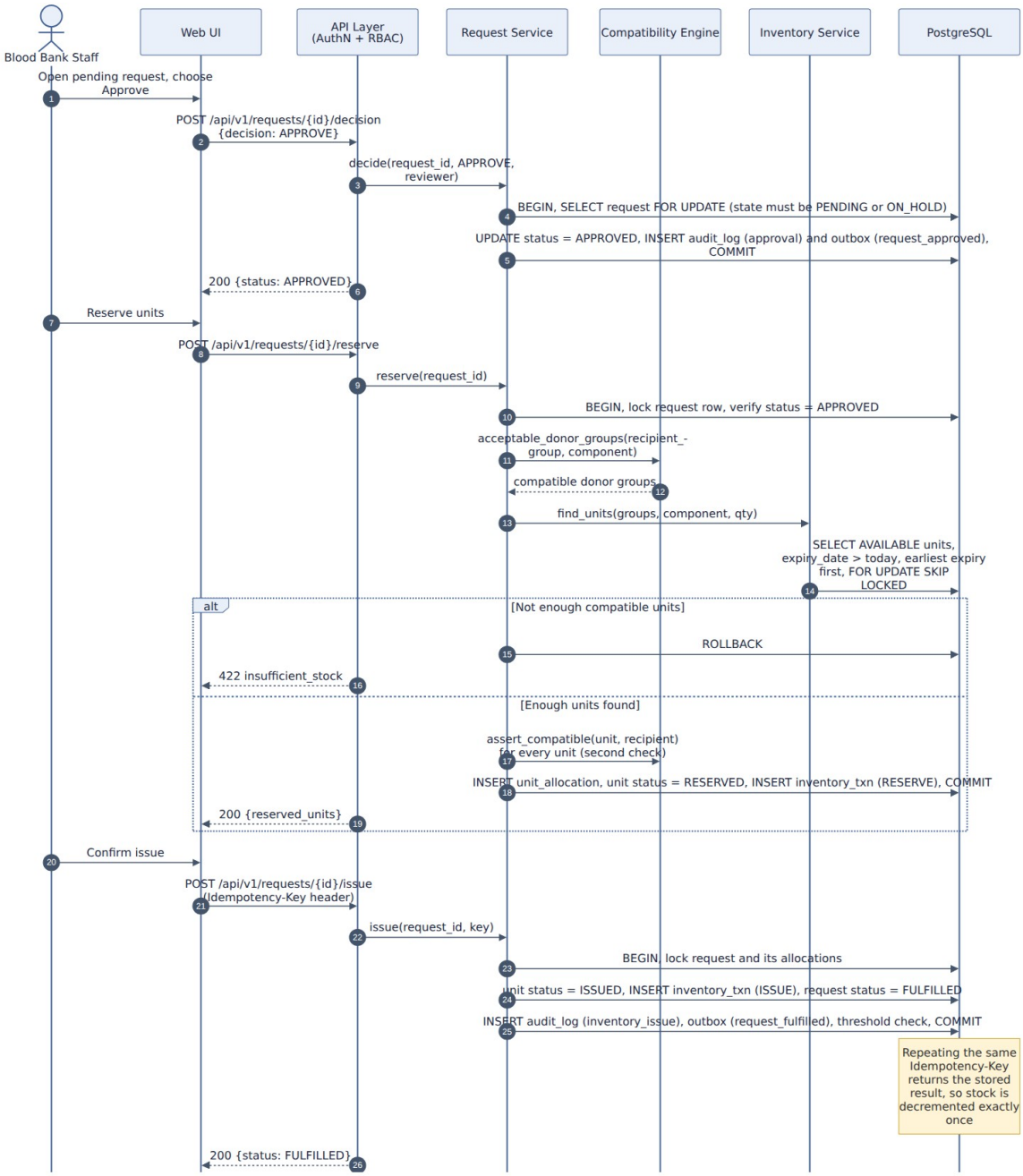



*Figure 7. Sequence diagram: approve, reserve and issue*

### *Flow 3: Daily expiry job and notification delivery*

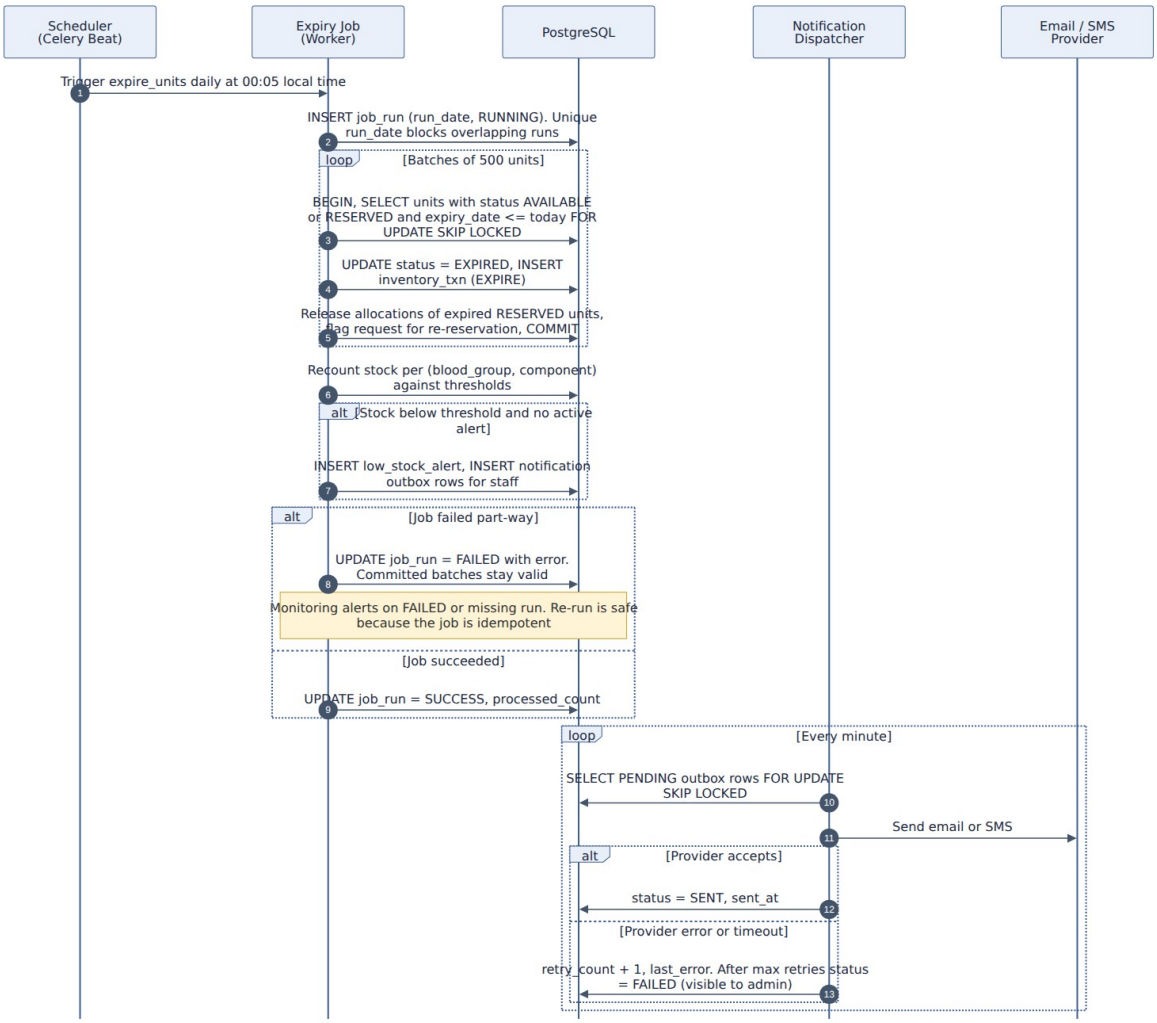



*Figure 8. Sequence diagram: daily expiry job, low-stock alert and notification dispatch*

### *Flow 4: Login, failure handling and session expiry*

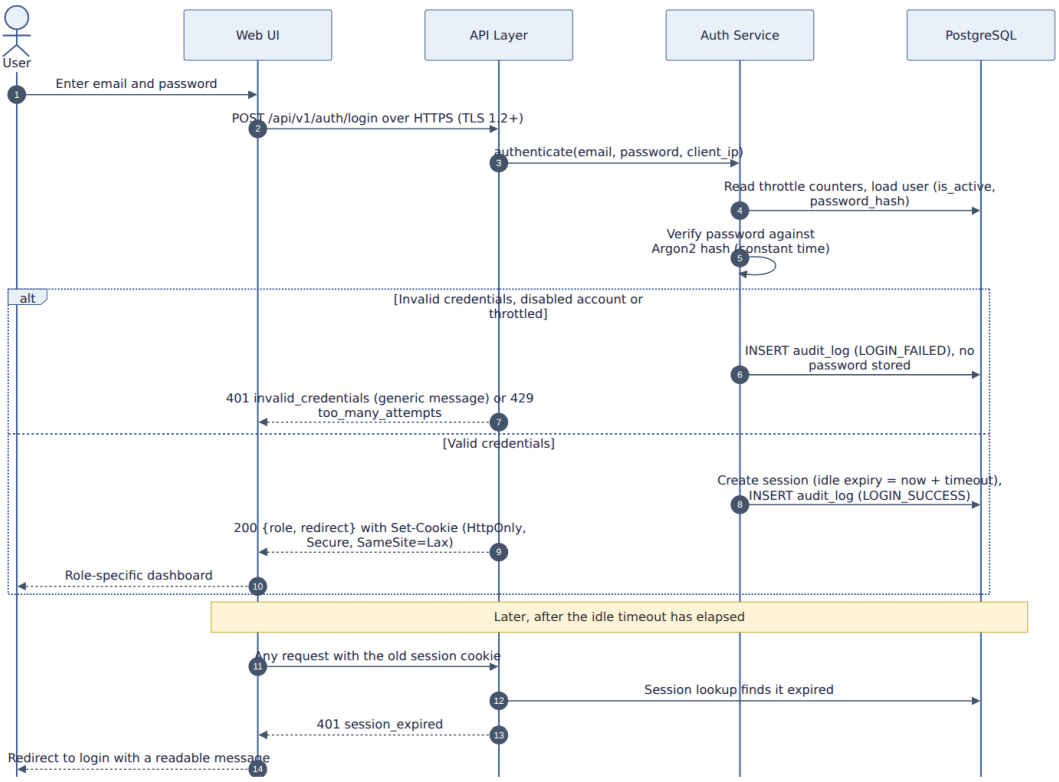



*Figure 9. Sequence diagram: login and session expiry*

## 4.3 API Design

### *Conventions*

- Base path /api/v1, JSON bodies, UTF-8, HTTPS only. Dates use ISO 8601; timestamps are UTC.
- Authentication is the session cookie set at login. State-changing calls also send the CSRF token header.
- List endpoints are paginated (page, page_size, maximum 100).
- Errors use one envelope (Section 4.4): { "error": { "code", "message", "details", "correlation_id" } }.
- Status codes: 400 validation, 401 not authenticated or session expired, 403 role not permitted, 404 not found or not owned, 409 conflict or invalid state, 422 business rule failed, 429 throttled, 500 server error, 503 unavailable.

### *Endpoint summary*

| Method | Endpoint | Allowed roles | Requirement |
|---|---|---|---|
| **Auth and Profile (M1)** | | | |
| POST | /auth/register | Public (Donor or Recipient) | F-001 |
| POST | /auth/login | Public | F-002 |
| POST | /auth/logout | Any signed-in user | F-002 |
| GET, PATCH | /users/me | Any signed-in user | F-004 |
| **Donor and Donation (M2)** | | | |
| GET | /donation-slots?date= | Donor, Staff | F-010 |
| POST, GET | /donors/me/appointments | Donor | F-010, F-012, F-013 |
| POST | /donors/me/appointments/{id}/cancel | Donor | F-010 |
| GET | /donors/me/donations | Donor | F-011 |
| GET | /donors/{id} | Staff | F-011 |

| Method | Endpoint | Allowed roles | Requirement |
|---|---|---|---|
| POST | /donations | Staff | F-011, F-013 |
| **Inventory (M3)** | | | |
| POST | /inventory/units | Staff | F-020, F-021 |
| GET | /inventory/units?blood_group=&component=&status= | Staff | F-023 |
| GET | /inventory/availability?blood_group=&component= | Staff, Recipient (aggregate) | F-023 |
| POST | /inventory/units/{id}/discard | Staff | F-022 |
| POST | /inventory/units/{id}/return | Staff | F-022 |
| **Requests (M4)** | | | |
| POST | /requests | Recipient | F-030 |
| GET | /requests, /requests/{id} | Recipient (own), Staff (all) | F-033 |
| POST | /requests/{id}/decision | Staff | F-031 |
| POST | /requests/{id}/reserve | Staff | F-032, F-034 |
| POST | /requests/{id}/issue | Staff | F-032 |
| **Notifications and Reports (M5)** | | | |
| GET | /notifications | Any signed-in user (own) | F-040, F-041 |
| GET | /reports/{donations\|inventory\|requests\|issued-units}?from=&to= | Staff, Admin | F-042 |
| **Administration (M6)** | | | |
| GET, POST | /admin/users | Admin | F-050 |
| PATCH | /admin/users/{id} | Admin | F-050 |
| GET, PUT | /admin/stock-thresholds | Admin | F-041 |
| GET | /admin/audit-logs?actor=&action=&from=&to= | Admin | F-051 |
| GET | /admin/notification-failures | Admin | F-040 |
| GET | /healthz | Public (no sensitive data) | NF-002 |

### *Interface definitions*

The following interfaces are specified in full for four components: Auth, Donor and Donation, Inventory, and Request and Fulfillment.

### *Auth: POST /api/v1/auth/login*

| | |
|---|---|
| **Purpose** | Authenticate a user and start a session (F-002). |
| **Request** | { "email": "staff1@bbms.example", "password": "********" } |
| **Success 200** | { "user": { "id": "u-8f2c", "name": "A. Rao", "role": "STAFF" },<br>"redirect": "/staff/dashboard",<br>"session_expires_in": 1200 }<br>Set-Cookie: sessionid=...; HttpOnly; Secure; SameSite=Lax |
| **Errors** | 400 validation_error (missing field)<br>401 invalid_credentials (same message for unknown user, wrong password, disabled account)<br>429 too_many_attempts (with Retry-After) |
| **Audit** | LOGIN_SUCCESS or LOGIN_FAILED, with actor if known, IP and time. The password is never logged. |

### *Donor: POST /api/v1/donors/me/appointments*

| | |
|---|---|
| **Purpose** | Book a donation slot for the signed-in donor (F-010, F-012, F-013). |
| **Request** | { "slot_id": "slot-2026-10-20-0930" } |

| | |
|---|---|
| **Success 201** | { "appointment_id": "apt-51d0",<br>"slot": { "starts_at": "2026-10-20T09:30:00+05:30" },<br>"status": "BOOKED" } |
| **Errors** | 401 unauthenticated, 403 forbidden (not a donor)<br>404 slot_not_found<br>409 slot_full, 409 duplicate_appointment<br>422 ineligible_donor |
| **422 detail** | { "error": { "code": "ineligible_donor",<br>"message": "You cannot book yet.",<br>"details": { "failed_rules": [ { "rule": "MIN_INTERVAL",<br>"message": "56 days are required between whole-blood donations.",<br>"next_eligible_date": "2026-11-20" } ] } } } |

### *Inventory: GET /api/v1/inventory/availability*

| | |
|---|---|
| **Purpose** | Search currently available blood by group and component (F-023). Recipients receive aggregate counts only; staff use /inventory/units for unit-level detail. |
| **Request** | GET /api/v1/inventory/availability?blood_group=O-&component=RBC |
| **Success 200** | { "results": [ { "blood_group": "O-", "component": "RBC",<br>"available_units": 14, "nearest_expiry": "2026-10-21" } ] } |
| **Rule** | Only units with status AVAILABLE and expiry_date after today are counted. |
| **Errors** | 400 invalid_blood_group or invalid_component, 401 unauthenticated, 403 forbidden (Donor, Admin) |

### *Requests: POST /api/v1/requests*

| | |
|---|---|
| **Purpose** | Submit a blood request (F-030). |
| **Request** | { "patient_name": "R. Kumar", "patient_blood_group": "A-",<br>"component": "RBC", "units_needed": 2, "urgency": "URGENT",<br>"hospital": "City General", "required_by": "2026-10-06T18:00:00+05:30",<br>"contact_phone": "+91XXXXXXXXXX" } |
| **Success 201** | { "request_id": "REQ-2026-000123", "status": "PENDING",<br>"created_at": "2026-10-04T10:12:44Z" } |
| **Errors** | 400 validation_error with field-level details, 401 unauthenticated, 403 forbidden (not a Recipient) |

### *Requests: POST /api/v1/requests/{id}/decision*

| | |
|---|---|
| **Purpose** | Staff approve, reject or hold a request (F-031). |
| **Request** | { "decision": "REJECT", "reason": "Patient details could not be verified" } |
| **Success 200** | { "request_id": "REQ-2026-000123", "status": "REJECTED",<br>"reviewed_by": "u-8f2c", "reviewed_at": "2026-10-04T10:40:02Z" } |
| **Errors** | 400 reason_required (REJECT or HOLD without reason), 403 forbidden, 404 not_found, 409 invalid_state_transition |
| **Side effects** | Audit record, and an outbox notification to the requester, in the same transaction. |

### *Requests: POST /api/v1/requests/{id}/reserve and /issue*

| | |
|---|---|
| **Purpose** | Reserve compatible units for an approved request, then issue them (F-032, F-034). The issue call requires an Idempotency-Key header. |
| **Reserve 200** | { "request_id": "REQ-2026-000123",<br>"reserved_units": [ { "unit_code": "BU-000412", "blood_group": "A-",<br>"component": "RBC", "expiry_date": "2026-10-28" } ] } |
| **Issue 200** | { "request_id": "REQ-2026-000123", "status": "FULFILLED",<br>"issued_units": [ { "unit_code": "BU-000412", "blood_group": "A-" } ] } |

| | |
|---|---|
| **Errors** | 400 missing_idempotency_key, 403 forbidden, 404 not_found<br>409 invalid_state (request not APPROVED, or not reserved), 409 unit_unavailable (reserved unit expired or discarded)<br>422 insufficient_stock, 422 incompatible_blood_type (safety re-check failed; transaction rolled back) |
| **Guarantee** | Repeating an issue call with the same key returns the stored result and does not decrement stock again. |

## 4.4 Error Handling, Logging & Monitoring

### *Error handling*

All failures return the same envelope, and no stack trace or internal detail is ever sent to the client:

```
{ "error": { "code": "slot_full",
   "message": "That time slot is fully booked. Please choose another.",
   "details": {},
   "correlation_id": "c4e1-93aa" } }
```

| Situation | HTTP | Handling |
|---|---|---|
| Invalid or missing input | 400 | Field-level messages; nothing is saved. |
| No session or session expired | 401 | UI redirects to login with a readable message. |
| Role not permitted | 403 | Generic message. Access-denied on sensitive operations is audited. |
| Not found, or not owned by the caller | 404 | Same response for both, to prevent ID enumeration. |
| Conflict or invalid state | 409 | Transaction rolled back; the UI refreshes the record and explains. |
| Business rule failed (ineligible donor, incompatible blood type, insufficient stock) | 422 | Names the rule that failed so staff and donors know what to do. |
| Too many attempts | 429 | Retry-After header; event logged. |
| Unexpected error | 500 | Transaction rolled back, generic message with correlation_id; full detail only in the application log; alert raised. |
| Database unavailable | 503 | Friendly maintenance page; health probe fails; alert raised. |
| Notification provider failure | n/a | Asynchronous. Row stays in outbox, retried with back-off up to 5 times, then marked FAILED and listed for Admin. Never blocks the user action. |
| Scheduled job failure | n/a | job_run marked FAILED with error text; committed batches stay valid; alert; safe to re-run. |

### *Logging*

There are two separate streams.

- **Application log:** structured JSON with timestamp, level, module, correlation_id and request path. Passwords, tokens, session ids and clinical or contact details are filtered out.
- **Audit log:** a protected database table (ADR-008), retained at least 1 year (NF-004). It records actor, role, action, entity, outcome, IP, correlation id and UTC time.

Audit event catalogue (F-051, SR-004): LOGIN_SUCCESS, LOGIN_FAILED, SESSION_EXPIRED, USER_CREATED, USER_DISABLED, ROLE_CHANGED, DONOR_UPDATED, DONATION_RECORDED, UNIT_CREATED, UNIT_RESERVED, UNIT_ISSUED, UNIT_EXPIRED, UNIT_DISCARDED, UNIT_RETURNED, REQUEST_SUBMITTED, REQUEST_APPROVED, REQUEST_REJECTED, REQUEST_HELD, REQUEST_FULFILLED, THRESHOLD_CHANGED, ACCESS_DENIED (sensitive endpoints).

### *Monitoring*

The thresholds below are initial values for the team to tune during testing.

| Signal | Alert condition | Source | Req |
|---|---|---|---|
| 95th percentile latency, search and dashboard | Above 3 seconds for 5 minutes | Request metrics | NF-001 |
| Server error rate (5xx) | Above 1% for 5 minutes | Request metrics | NF-002 |
| Availability probe on /healthz | 3 consecutive failures | External probe, 1 min | NF-002 |
| Expiry job | No SUCCESS run in 26 hours, or status FAILED | job_run table | F-024 |
| Notification outbox | More than 100 pending, or failure rate above 5% in an hour | notification table | F-040 |
| Backup | No successful backup in 25 hours | Backup job status | NF-006 |
| Failed logins | More than 20 per minute from one address | Audit log | SR-004 |
| Database capacity | Connections or disk above 80% | Database metrics | NF-002 |

**Backup and recovery (NF-006).** A full database dump runs daily at 02:00 and is copied, encrypted, to offsite storage, giving a recovery point of at most 24 hours. A written restore runbook and a periodic restore drill on a spare host verify that service can be recovered within the 4-hour objective.

## 4.5 UX Design

The UI is a responsive web application with one dashboard per role (SRS 3.1). Screens are simple and forgiving because staff have varied technical experience.

| Role | Dashboard content | Primary actions |
|---|---|---|
| Donor | Next appointment, eligibility status with next eligible date, donation history. | Book or cancel appointment, edit profile. |
| Recipient | Availability by group and component (counts), own requests with status and timestamps. | Submit request, track request. |
| Staff | Pending requests ordered by urgency, low-stock alerts, units expiring soon, today's appointments. | Record donation, add unit, review request, reserve and issue, search stock. |
| Administrator | Account list, system health summary, recent audit events, notification failures. | Manage staff accounts and roles, set thresholds, run reports, search audit log. |

### *Design principles*

- **Accessibility (NF-005):** every control has a visible label; full keyboard navigation with visible focus; sufficient colour contrast; errors announced to screen readers with aria-live; the target is WCAG 2.1 AA.
- **Status never relies on colour alone.** Each status chip pairs colour with text and an icon: Available (green, check), Reserved (blue, lock), Issued (grey), Expiring within 7 days (amber, clock), Expired (red, cross).
- **Readable errors:** plain language that says what went wrong and what to do next, displayed next to the field concerned and summarised at the top of the form.
- **Safe actions:** irreversible actions (issue, discard) use a confirmation step that restates the unit codes and the recipient group.
- **Responsive layout:** Bootstrap grid; tables collapse to cards on narrow screens; touch targets of at least 44 px.
- **Scanner friendly:** the unit-code field is auto-focused on staff unit screens so a barcode/QR scanner acting as a keyboard fills it directly.

## 4.6 Open Issues & Next Steps

### *Open issues*

| ID | Issue | Proposed resolution |
|---|---|---|
| OI-1 | SRS F-034 gives red-cell style rules. Plasma and platelets follow different ABO rules. | Medical reviewer to confirm. The engine is already parameterised by component, so stricter rule sets can be added without touching callers. |
| OI-2 | The 56-day interval applies to whole blood. Other donation types may need different intervals. | Confirm with medical stakeholder; make the interval configurable per donation type. |
| OI-3 | Expiry boundary: SRS says units expire "when their expiry date is reached". This design treats a unit as expired on the expiry date itself. | Product Owner and medical reviewer to confirm; STP boundary tests (Section 5.2) depend on it. |
| OI-4 | Assumption A-3: Admin does not perform operational actions. | Product Owner to confirm. If Admin should also approve or issue, update the matrix in 3.9.2 and the STP permission tests. |
| OI-5 | One role per account. A person who is both donor and recipient needs two accounts. | Acceptable for v1.0; consider multiple roles later. |
| OI-6 | SRS defines JWT in its acronyms but allows sessions or tokens. This design chooses sessions (ADR-004). | Team to confirm. Revisit if an external hospital API client is required. |
| OI-7 | Notification provider (email and SMS) not selected. | Choose a provider or keep console/mock for the project demonstration. |
| OI-8 | Default stock thresholds, slot capacities and backup retention are not defined. | Staff representatives to supply initial values; configurable by Admin. |
| OI-9 | SRS refers to "supported interfaces" to external hospital systems but defines none. | Out of scope for v1.0; the REST API is the future integration point. |

### *Next steps*

1. Team review of this document and resolution of OI-1 to OI-4 with the Product Owner.
2. Copy the DS IDs from Section 3.8 into the Design Spec column of the SRS RTM.
3. Freeze the API contract (OpenAPI file) and share the Postman collection with the Test Engineer.
4. Implement the Compatibility Engine, Eligibility Engine and their exhaustive unit tests first (lowest cost, highest safety value).
5. Build the vertical slices in this order: login and RBAC, inventory, donation, request and fulfillment, expiry job, notifications, reports, admin and audit.
6. Create the seed_test_data command and the notification mock before STP test execution.
7. Run JMeter against search and dashboard endpoints and perform the backup restore drill before UAT.

# 5. Appendices

## 5.1 Glossary

| Term | Meaning |
|---|---|
| **ABO / Rh** | The main blood group system. A, B, AB and O are the ABO groups; "+" or "-" is the Rh factor. |
| **Component** | The part of donated blood stored and issued separately: whole blood, red blood cells (RBC), plasma, platelets. |
| **ADR** | Architecture Decision Record: a short record of a design decision, alternatives and rationale. |
| **Allocation** | The link between a blood request and a specific reserved unit. |
| **Audit log** | Append-only record of logins and critical changes, with actor, action, time and affected record. |
| **FEFO** | First-Expired, First-Out: issue the unit that expires soonest. |
| **Idempotency key** | A client-supplied token that makes a repeated call return the original result instead of repeating the action. |
| **Ledger (inventory)** | Append-only list of every stock movement (reserve, issue, expire, discard, return). |
| **Outbox** | A database table of messages to send, written in the same transaction as the event that caused them. |
| **RBAC** | Role-Based Access Control: permissions determined by a user's role. |
| **RPO / RTO** | Recovery Point Objective (maximum data loss) and Recovery Time Objective (maximum time to restore service). |
| **STRIDE** | Threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. |
| **TLS** | Transport Layer Security, the encryption protocol behind HTTPS. |
| **Unit** | One collected bag or portion of blood or a blood component, tracked individually by a unique code. |

## 5.2 References

- Team 9, Blood Bank Management System Software Requirements Specification, v1.0, 03-09-2026.
- Team 9, Blood Bank Management System Software Test Plan, v1.0, 17-09-2026.
- ISO/IEC/IEEE 42010, Systems and software engineering: Architecture description.
- NIST SP 800-160, Systems Security Engineering.
- NIST SP 800-63B, Digital Identity Guidelines: Authentication and Lifecycle Management (password and session guidance).
- OWASP Application Security Verification Standard (ASVS) and OWASP Top 10.
- Django and Django REST Framework documentation (security, sessions, password hashing, transactions).
- PostgreSQL documentation (explicit locking, partial indexes).

## 5.3 Tools

- Diagrams in this document: Graphviz (component and deployment views) and Mermaid (ER, state and sequence diagrams). PlantUML or draw.io can be used to maintain them.
- API specification and documentation: OpenAPI with Swagger UI (drf-spectacular), Postman.
- Testing: Pytest, Selenium, Postman, JMeter (STP Section 6).
- Source control and defect tracking: Git, GitHub, GitHub Issues.